

程序，因而也称为高级语言程序员，所看到的虚拟机器称为高级语言虚拟机；最终用户则工作在最上面的应用程序层。

1.4 本章小结

计算机在控制器的控制下完成数据处理、数据存储和数据传输三个基本功能，因而它由完成相应功能的控制器、运算器、存储器、输入 / 输出设备组成。在计算机内部，指令和数据都用二进制表示，两者在形式上没有任何差别，都是 0/1 序列，都存放在存储器中，按地址访问。计算机采用“存储程序”方式进行工作。指令格式中包含操作码字段和地址码字段等，地址码可以是主存单元号，也可以是通用寄存器编号，用于指出操作数所在的主存单元或通用寄存器。

计算机系统以逐层向上抽象的方式构成，通过向上层用户提供一个抽象的简洁接口将较低层次的实现细节隐藏起来。系统中软件和硬件之间的抽象层就是指令集体系结构（ISA）。硬件和软件相辅相成，缺一不可，两者都可用来实现逻辑功能。

计算机完成一个任务的大致过程如下：用某种程序设计语言编制源程序；用语言处理程序将源程序翻译成机器语言目标程序；将目标程序中的指令和数据装入内存，然后从第一条指令开始执行，直到程序中的指令全部执行完。每条指令的执行包括取指令、指令译码、PC 增量、取操作数、运算、送结果等操作。

习题

1. 给出以下概念的解释说明。

中央处理器（CPU）	算术逻辑部件（ALU）	通用寄存器	程序计数器（PC）
指令寄存器（IR）	控制器	主存储器（MM）	总线
主存地址寄存器（MAR）	主存数据寄存器（MDR）	指令操作码	指令地址码
微操作	控制信号	时钟信号	时钟周期
机器指令	高级程序设计语言	汇编语言	机器语言
机器级语言	源程序	目标程序	编译程序
解释程序	汇编程序	语言处理系统	设备控制器
指令集体系结构（ISA）	微体系结构	应用程序二进制接口（ABI）	最终用户
系统管理员	应用程序员	系统程序员	透明

2. 简单回答下列问题。

选择

- (1) 冯·诺依曼计算机由哪几部分组成？各部分的功能是什么？
- (2) 什么是“存储程序”工作方式？
- (3) 一条指令的执行过程包含哪几个阶段？
- (4) 如何划分计算机系统的层次结构？
- (5) 什么是程序的未定义行为？什么是程序的未指定行为？什么是程序的实现定义行为？
- (6) 计算机系统的用户可分为哪几类？每类用户工作在哪个层次？



- (7) 应用程序二进制接口 (ABI) 和应用程序接口 (API) 各自与哪类计算机系统用户关系最密切 (即哪类用户直接使用 ABI 和 API 标准)?
3. 假定你的朋友不太懂计算机, 请用简单通俗的语言向你的朋友介绍计算机系统是如何工作的。
4. 你对计算机系统的哪些部分最熟悉, 哪些部分最不熟悉? 你最想进一步了解哪些部分的细节内容?
5. 图 1.1 所示的模型机 (采用图 1.2 所示的指令格式) 的指令系统中, 除了有 `mov (op=0000)`、`add (op=0001)`、`load (op=1110)` 和 `store (op=1111)` 指令外, R 型指令还有 `sub (op=0010)` 和 `mul (op=0011)` 等指令, 请仿照图 1.3 给出求解表达式 “ $z=(x-y)*y;$ ” 所对应的指令序列 (包括机器代码和对应的汇编指令) 以及在主存中存放的内容, 并仿照图 1.5 给出每条指令的执行过程以及所包含的微操作。



符号扩展 零标志 ZF 溢出标志 OF 符号标志 SF 进 / 借位标志 CF

2. 简单回答下列问题。
- (1) 为什么计算机内部采用二进制表示信息？既然计算机内部所有信息都用二进制表示，为什么还要用到十六进制或八进制数？

(2) 常用的定点数编码方式有哪几种？通常它们各自用来表示什么信息？

(3) 为什么现代计算机中大多用补码表示带符号整数？

(4) 在浮点数的基数和总位数一定的情况下，浮点数的表示范围和精度分别由什么决定？两者如何相互制约？

(5) 为什么要对浮点数进行规格化？有哪两种规格化操作？

(6) 为什么计算机处理汉字时会涉及不同的编码（如输入码、内码、字模码）？说明这些编码中哪些用二进制编码，哪些不用二进制编码，为什么？
3. 实现下列各数的转换。
- (1) $(28.125)_{10} = (\quad)_2 = (\quad)_8 = (\quad)_{16}$

(2) $(1001111.01)_2 = (\quad)_{10} = (\quad)_8 = (\quad)_{16}$

(3) $(8A.B)_{16} = (\quad)_{10} = (\quad)_2$
4. 假定机器数为 8 位（1 位符号，7 位数值），写出下列各二进制的原码表示。
- $+0.10011, -0.10011, +1.0, -1.0, +0.01101, -0.01101, +0, -0$
5. 假定机器数为 8 位（1 位符号，7 位数值），写出下列各二进制的补码和移码表示。
- $+10011, -10011, +1, -1, +1101, -1101, +0, -0$
6. 已知 $[x]_{\text{补}}$ ，求 x
- (1) $[x]_{\text{补}}=1100\ 0101$

(2) $[x]_{\text{补}}=1000\ 0000$

(3) $[x]_{\text{补}}=0111\ 0010$

(4) $[x]_{\text{补}}=1111\ 0010$
7. 某 32 位字长的机器中带符号整数用补码表示，浮点数用 IEEE 754 标准表示，寄存器 R1 和 R2 的内容分别为 0000 108BH 和 8080 108BH。不同指令对寄存器进行不同的操作，因而不同指令执行时寄存器内容对应的真值不同。假定执行下列运算指令时，操作数为寄存器 R1 和 R2 的内容，则 R1 和 R2 中操作数的真值分别为多少？
- (1) 无符号整数加法指令

(2) 带符号整数乘法指令

(3) 单精度浮点数减法指令
8. 假定机器 M 的字长为 32 位，用补码表示带符号整数。表 2.12 中第一列给出了在机器 M 上执行的 C 程序中的关系表达式，请参照已有的表栏内容完成表中后三栏内容的填写。

无符号提升，看成无符号

表 2.12 题 8 表

	关系表达式	运算类型	结果	说明
1	$0 == 0U$			
2	$-1 < 0$			
3	$-1 < 0U$			
4	$2147483647 > -2147483647 - 1$	无符号整数	0	$11\cdots 1B\ (2^{32}-1) > 00\cdots 0B(0)$
5	$2147483647U > -2147483647 - 1$	带符号整数	1	$011\cdots 1B\ (2^{31}-1) > 100\cdots 0B\ (-2^{31})$
6	$2147483647 > (\text{int})\ 2147483648U$			
7	$-1 > -2$			
8	$(\text{unsigned})\ -1 > -2$			

float S(1) + E(8) + M(23)

小数 → 二进制

乘2, 取整数部分, 小数部分继续×2

double S(1) + E(11) + M(52)

偏置: 127/1023

9. 在 32 位计算机中运行一个 C 程序, 该程序中出现了以下变量的初值, 请写出它们对应的机器数 (用十六进制表示)。

- (1) int x = -32768 (2) short y = 522 × (3) unsigned z = 65530
(4) char c = '@' × (5) float a = -1.1 × (6) double b = 10.5

10. 在 32 位计算机中运行一个 C 程序, 该程序中出现了某些变量, 已知这些变量在某一时刻的机器数 (用十六进制表示) 如下, 请写出它们对应的真值。

- (1) int x: FFFF 0006H (2) short y: DFFCH (3) unsigned z: FFFF FFFAH
(4) char c: 2AH (5) float a: C448 0000H (6) double b: C024 8000 0000 0000H

11. 以下给出的是一些字符串变量的机器码, 请根据 ASCII 码定义写出对应的字符串。

- (1) char *mystring1: 68H 65H 6CH 6CH 6FH 2CH 77H 6FH 72H 6CH 64H 0AH 00H
(2) char *mystring2: 77H 65H 20H 61H 72H 65H 20H 68H 61H 70H 70H 79H 21H 00H

12. 以下给出的是一些字符串变量的初值, 请写出对应的机器码。

- (1) char *mystring1 = "/myfile" (2) char *mystring2 = "OK, good!"

13. 已知 C 语言中的按位异或运算 (XOR) 用符号 “^” 表示。对于任意一个位序列 a , $a^a = 0$, C 程序可利用该特性实现两个数值交换的功能。以下是相应的 C 函数:

```
1 void xor_swap(int *x, int *y)
2 {
3     *y = *x ^ *y; /* 第一步 */
4     *x = *x ^ *y; /* 第二步 */
5     *y = *x ^ *y; /* 第三步 */
6 }
```

假定执行该函数时 $*x$ 和 $*y$ 的初始值分别为 a 和 b , 即 $*x = a$ 且 $*y = b$, 请给出每一步执行结束后, x 和 y 各自指向的存储单元中的内容分别是什么?

14. 假定某个实现数组元素倒置的函数 `reverse_array()` 时调用了第 13 题中给出的 `xor_swap()` 函数:

```
1 void reverse_array(int a[], int len)
2 {
3     int left, right = len - 1;
4     for (left = 0; left <= right; left++, right--)
5         xor_swap(&a[left], &a[right]);
6 }
```

当 len 为偶数时, `reverse_array()` 函数执行没问题。但当 len 为奇数时, 函数执行结果不正确。请问: 当 len 为奇数时会出现什么问题? 最后一次循环中 $left$ 和 $right$ 各取什么值? 最后一次循环中调用 `xor_swap()` 函数后返回值是什么? 怎样改动 `reverse_array()` 函数就可消除该问题?

15. 假设表 2.13 中的 x 和 y 是某 C 程序中的 `char` 型变量, 根据 C 语言中的按位运算和逻辑运算的定义, 填写表 2.13, 要求用十六进制形式填写。

表 2.13 题 15 表

x	y	x^y	$x \& y$	$x y$	$\sim x \sim y$	$x \& !y$	$x \& \& y$	$x y$	$!x !y$	$x \& \& \sim y$
0x3E	0xAB									
0xC8	0xF0									
0x8F	0x70									
0x09	0x55									



16. 对于一个 n ($n \geq 8$) 位的变量 x , 请根据 C 语言中按位运算的定义, 写出满足下列要求的 C 语言表达式。

- (1) x 的最高有效字节不变, 其余各位全变为 0。
- (2) x 的最低有效字节不变, 其余各位全变为 0。
- (3) x 的最低有效字节全变为 0, 其余各位取反。
- (4) x 的最低有效字节全变 1, 其余各位不变。

17. 以下是一个由反汇编器生成的一段针对某小端方式处理器的机器级代码表示文本, 其中, 最左边是指令所在的存储单元地址, 冒号后是指令的机器码, 最右边是对应汇编指令。已知反汇编输出的机器数用补码表示, 给出指令代码中画线部分机器数对应的真值。

```

80483d2: 81 ec b8 01 00 00      sub    $0x1b8, %esp
80483d8: 8b 55 08                  mov    0x8(%ebp), %edx
80483db: 83 c2 14                add    $0x14, %edx
80483de: 8b 85 58 fe ff ff      mov    0xfffffe58(%ebp), %eax
80483e4: 03 02                    add    (%edx), %eax
80483e6: 89 85 74 fe ff ff      mov    %eax, 0xfffffe74(%ebp)
80483ec: 8b 55 08                  mov    0x8(%ebp), %edx
80483ef: 83 c2 44                add    $0x44, %edx
80483f2: 8b 85 c8 fe ff ff      mov    0xfffffec8(%ebp), %eax
80483f8: 89 02                    mov    %eax, (%edx)
80483fa: 8b 45 10                mov    0x10(%ebp), %eax
80483fd: 03 45 0c                add    0xc(%ebp), %eax
8048400: 89 85 ec fe ff ff      mov    %eax, 0xfffffeec(%ebp)
8048406: 8b 45 08                  mov    0x8(%ebp), %eax
8048409: 83 c0 20                add    $0x20, %eax

```

18. 假设以下 C 语言函数 `compare_str_len()` 用来判断两个字符串的长度, 当字符串 `str1` 的长度大于 `str2` 的长度时函数返回值为 1, 否则为 0。

无符号数没有
负数, 超出范围
会对 2 取模
eg. 16-32 = 2³²-16

```

1 int compare_str_len(char *str1, char *str2)
2 {
3     return strlen(str1) - strlen(str2) > 0;
4 }

```

已知 C 标准库函数 `strlen()` 原型声明为 “`size_t strlen(const char *s);`”, 其中, `size_t` 定义为 `unsigned int` 类型。请问: 函数 `compare_str_len()` 在什么情况下返回结果不正确? 为什么? 为使函数正确返回结果, 应如何修改代码?

19. 考虑以下 C 程序代码:

```

1 int func1(unsigned word)
2 {
3     return (int) (( word <<24) >> 24);
4 }
5
6 int func2(unsigned word)
7 {
8     return ( (int) word <<24 ) >> 24;
9 }

```



假设在一个 32 位机器上执行这些函数，该机器使用二进制补码表示带符号整数。无符号整数采用逻辑移位，带符号整数采用算术移位。请填写表 2.14，并说明函数 func1() 和 func2() 的功能。

表 2.14 题 19 表

w		func1(w)		func2(w)	
机器数	值	机器数	值	机器数	值
	127				
	128				
	255				
	256				

20. 填写表 2.15，注意对比无符号整数和带符号整数的乘法结果及截断操作前后的结果。

表 2.15 题 20 表

模式	x		y		x × y (截断前)		x × y (截断后)	
	机器数	值	机器数	值	机器数	值	机器数	值
无符号	1101		0101					
带符号	1101		0101					
无符号	0011		1111					
带符号	0011		1111					
无符号	1111		1111					
带符号	1111		1111					

21. 以下 C 函数 arith() 是直接 C 语言写的，而 optarith() 是对 arith() 函数以某个确定的 M 和 N 编译生成的机器代码反编译生成的。根据 optarith() 推断函数 arith() 中 M 和 N 的值各是多少，并推断编译器对 arith() 函数做了哪些优化。

```
1 #define M
2 #define N
3 int arith(int x, int y)
4 {
5     int result = 0 ;
6     result = x*M + y/N;
7     return result;
8 }
9
10 int optarith ( int x, int y)
11 {
12     int t = x;
13     x <<= 4;
14     x -= t;
15     if ( y < 0 ) y += 3;
16     y>>=2;
17     return x+y;
18 }
```

存在问题：
y < 0 时
 $y \gg 2 \neq y/4$
eg. $y = -1$. $y \gg 2 = -1$. $y/4 = 0$
移位规则：
① 带符号数. 算术右移.
② sign=0 或 unsigned. 直接移
③ sign=1. 需校正. $y+2^{10}-1$. 再移位

函数右移添加偏置常数

22. 下列几种情况所能表示的数的范围是什么？
(1) 16 位无符号整数
(2) 16 位原码定点小数

min 全0
max 全1
1/68

移码值 = 真值 + 偏置常数

(3) 16 位移码定点整数 (偏置常数为 32 768)

(4) 16 位移码定点整数 (偏置常数为 32 767)

(5) 16 位补码定点整数

1 0 15 0 15

(6) 下述格式的浮点数 (基为 2, 移码的偏置常数为 128)

数符	阶码	尾数
----	----	----

1 位 8 位移码 7 位原码小数数值部分

规格化尾数最高位为 1

0.1...

100 0x2¹²⁷
-100 -0x2¹²⁷

★ 23/ 以 IEEE 754 单精度浮点数格式表示下列十进制数。

+1.75, +19, -1/8, 258

编码阶 = 真值阶 + 偏置

3.3. 1.1 -3.3

0舍1入

24/ 设一个变量的值为 4098, 要求分别用 32 位补码整数和 IEEE 754 单精度浮点格式表示该变量 (结果用十六进制形式表示), 并说明哪段二进制位序列在两种表示中完全相同, 为什么会相同?

25. 设一个变量的值为 -2 147 483 647, 要求分别用 32 位补码整数和 IEEE 754 单精度浮点格式表示该变量 (结果用十六进制形式表示), 并说明哪种表示其值完全精确, 哪种表示是近似值 (提示: 2 147 483 647 = 2³¹ - 1)。

26. 表 2.16 给出了有关 IEEE 754 浮点格式表示中一些重要的非负数的取值, 表中已经有最大规格化数的相应内容, 要求填入其他浮点格式的相关内容。

表 2.16 题 26 表

项目	阶码	尾数	单精度		双精度	
			以 2 的幂次表示的值	以 10 的幂次表示的值	以 2 的幂次表示的值	以 10 的幂次表示的值
0						
1						
最大规格化数	11111110	1...11	(2-2 ⁻²³) × 2 ¹²⁷	3.4 × 10 ³⁸	(2-2 ⁻⁵²) × 2 ¹⁰²³	1.8 × 10 ³⁰⁸
最小规格化数						
最大非规格化数						
最小非规格化数						
+ ∞						
NaN						

27. 已知下列字符编码: A 为 100 0001, a 为 110 0001, 0 为 011 0000。求 E、e、f、7、G、Z、5 的 7 位 ACSII 码和在第一位前加入奇校验位后的 8 位编码。

★

28/ 假定在一个程序中定义了变量 x、y 和 i, 其中, x 和 y 是 float 型变量, i 是 16 位 short 型变量 (用补码表示)。程序执行到某一时刻, x = -251.25、y = 130.125、i = 1000, 它们都被写到了主存 (按字节编址), 其地址分别是 100、108 和 112。请分别画出在大端机器和小端机器上变量 x、y 和 i 中每个字节在主存中的存放位置。

和结构体一起考

photo.

29/ 对于图 2.8, 假设 n=8, 机器数 X 和 Y 的真值分别是 x 和 y。请按照图 2.8 的功能填写表 2.17 并给出对每个结果的解释。要求机器数用十六进制形式填写, 真值用十进制形式填写。

0x10E 补码 0x52

表 2.17 题 29 表

表示	X	x	Y	y	X+Y	x+y	OF	SF	CF	X-Y	x-y	OF	SF	CF
无符号	0x4F		0xAE											
带符号	0x4F		0xAE											
无符号	0x70		0xC2											
带符号	0x70		0xC2											



30. 在字长为 32 位的计算机上, 有一个 C 函数原型声明为 “int ch_mul_overflow(int x, int y);”, 该函数用于对两个 int 型变量 x 和 y 的乘积判断是否溢出, 若溢出则返回 1, 否则返回 0。请使用 64 位整型 long long 来编写该函数。

31. 对于 2.6.5 节中例 2.31 存在的整数溢出漏洞, 如果将其中的第 5 行改为以下两个语句:

```
unsigned long long arraysize=count*(unsigned long long)sizeof(int);
int *myarray = (int *) malloc(arraysize);
```

已知 C 标准库函数 malloc() 的原型声明为 “void *malloc(size_t size);”, 其中, size_t 定义为 unsigned int 类型, 则上述改动能否消除整数溢出漏洞? 若能则说明理由; 若不能则给出修改方案。

32. 已知一次整数加法、一次整数减法和一次移位操作都只需一个时钟周期, 一次整数乘法操作需要 10 个时钟周期。若 x 为一个整型变量, 现要计算 $55 * x$, 请给出一种计算表达式, 使所用时钟周期数最少。

33. 假设 x 为一个 int 型变量, 请给出一个用来计算 $x/32$ 的值的函数 div32。要求不能使用除法、乘法、模运算、比较运算、循环语句和条件语句, 可以使用右移、加法以及任何按位运算。

34. 无符号整数变量 ux 和 uy 的声明和初始化如下:

```
unsigned ux=x;
unsigned uy=y;
```

若 sizeof(int)=4, 则对于任意 int 型变量 x 和 y, 判断以下关系表达式是否永真。若永真则给出证明; 若不永真, 则给出结果为假时 x 和 y 的取值。

- | | |
|---|---|
| (1) $(x * x) \geq 0$ | (2) $(x - 1 < 0) \parallel x > 0$ |
| (3) $x < 0 \parallel -x \leq 0$ | (4) $x > 0 \parallel -x \geq 0$ |
| (5) $x \& 0xf1 = 15 \parallel (x \ll 28) < 0$ | (6) $x > y == (-x < -y)$ |
| (7) $-x + -y == -(x + y)$ | (8) $(int) (ux - uy) == -(y - x)$ |
| (9) $((x > 2) \ll 2) \leq x$ | (10) $x * 4 + y * 8 == (x \ll 2) + (y \ll 3)$ |
| (11) $x / 4 + y / 8 == (x >> 2) + (y >> 3)$ | (12) $x * y == ux * uy$ |
| (13) $x + y == ux + uy$ | (14) $x * -y + ux * uy == -x$ |

35. 变量 dx、dy 和 dz 的声明和初始化如下:

```
double dx = (double) x;
double dy = (double) y;
double dz = (double) z;
```

若 float 和 double 分别采用 IEEE 754 单精度和双精度浮点数格式, sizeof(int)=4, 则对于任意 int 型变量 x、y 和 z, 判断以下关系表达式是否永真。若永真则给出证明; 若不永真则给出结果为假时 x、y 和 z 的取值。

- | | |
|------------------------------------|--|
| (1) $dx * dx \geq 0$ | (2) $(double)(float) x == dx$ |
| (3) $dx + dy == (double) (x + y)$ | (4) $(dx + dy) + dz == dx + (dy + dz)$ |
| (5) $dx * dy * dz == dz * dy * dx$ | (6) $dx / dx == dy / dy$ |

36. 在 IEEE 754 浮点数运算中, 当结果的尾数出现什么形式时需要进行左规, 什么形式时需要进行右规? 如何进行左规, 如何进行右规?

37. 在 IEEE 754 浮点数运算中, 如何判断浮点运算的结果是否溢出?



38. 分别给出不能精确用 IEEE 754 单精度和双精度格式表示的最小正整数。

39. 采用 IEEE 754 单精度浮点数格式计算下列表达式的值。

(1) $0.75 + (-65.25)$ (2) $0.75 - (-65.25)$

40. 以下是函数 `fpower2()` 的 C 程序，用于计算 2^x 的浮点数表示，其中调用了函数 `u2f()`。`u2f()` 用于将一个无符号整数表示的 0/1 序列作为 float 类型返回。请填写 `fpower2()` 函数中的空白部分，以使其能正确计算结果。

```
1 float fpower2(int x) {
2
3     unsigned exp, frac, u;
4
5     if (x<_____) {      /* 值太小，返回 0.0 */
6         exp = _____;
7         frac = _____;
8     } else if (x<_____) {      /* 返回非规格化结果 */
9         exp = _____;
10        frac = _____;
11    } else if (x<_____) {      /* 返回规格化结果 */
12        exp = _____;
13        frac = _____;
14    } else {      /* 值太大，返回 +∞ */
15        exp = _____;
16        frac = _____;
17    }
18    u = exp << 23 | frac;
19    return u2f(u);
20 }
```

41. 以下是一组关于浮点数按位级进行运算的编程题目，其中用到一个数据类型 `float_bits`，它被定义为 `unsigned int` 类型。以下程序代码必须采用 IEEE 754 标准规定的运算规则，例如，舍入应采用就近舍入到偶数的方式。此外，代码中不能使用任何浮点数类型、浮点数运算和浮点常数，只能使用 `float_bits` 类型；不能使用任何复合数据类型，如数组、结构体和联合等；可以使用无符号整数或带符号整数的数据类型、常数和运算。要求编程实现以下功能并进行正确性测试。

(1) 计算变量 f 的绝对值 $|f|$ 。若 f 为 NaN，则返回 f ，否则返回 $|f|$ 。函数原型如下：

```
float_bits float_abs(float_bits f);
```

(2) 计算变量 f 的负数 $-f$ 。若 f 为 NaN，则返回 f ，否则返回 $-f$ 。函数原型如下：

```
float_bits float_neg(float_bits f);
```

(3) 计算 $0.5*f$ 。若 f 为 NaN，则返回 f ，否则返回 $0.5*f$ 。函数原型如下：

```
float_bits float_half(float_bits f);
```

(4) 计算 $2.0*f$ 。若 f 为 NaN，则返回 f ，否则返回 $2.0*f$ 。函数原型如下：

```
float_bits float_twice(float_bits f);
```



(5) 将 int 型变量 i 的位序列转换为 float 型位序列。函数原型如下：

```
float_bits float_i2f(int i);
```

(6) 将变量 f 的位序列转换为 int 型位序列。若 f 为非规格化数，则返回值为 0；若 f 是 NaN 或 $\pm \infty$ 或超出 int 型数可表示范围，则返回值为 0x8000 0000；若 f 带小数部分，则考虑舍入。
函数原型如下：

```
int float_f2i(float_bits f);
```



`%xmm3`”的功能相当于 `AVX` 的 `vsubss(%rdx), %xmm3, %xmm3`”的功能, 都是将 `XMM3` 中低 32 位表示的单精度浮点数减去 `RDX` 指向的 4B 长度存储单元中的单精度浮点数, 结果存放在 `XMM3` 寄存器中。

除了上述部分浮点处理指令以外, 还有位操作和浮点数大小比较等指令, 详细内容可参考 Intel 相关指令系统手册。

3.4 本章小结

3-8

任何一个 C 语言程序都要转换为对应机器所采用的指令集体系结构规定的机器代码才能执行。本章主要介绍 IA-32/x86-64 指令集体系结构的基础内容, 包括 IA-32/x86-64 支持的数据类型、寄存器组织、寻址方式、常用指令类型、指令格式和指令的功能, 从而为下一章介绍 C 语言程序在 IA-32/x86-64 架构上的机器级表示奠定基础。

习题

3-10

1. 给出以下概念的解释说明。

机器语言程序	机器指令	汇编语言	汇编指令
汇编语言程序	汇编助记符	汇编程序	反汇编程序
机器级代码	通用寄存器	变址寄存器	基址寄存器
栈指针寄存器	指令指针寄存器	标志寄存器	标志位(条件码)
寻址方式	立即寻址	寄存器寻址	存储器操作数
相对寻址	基址寻址	变址寻址	实地址模式
保护模式	有效地址	比例变址	非比例变址
比例系数(比例因子)	MMX 指令	SSE 指令集	SIMD
AVX 指令集	多媒体扩展寄存器	数据级并行处理指令	

2. 简单回答下列问题。

- (1) 一条机器指令通常由哪些字段组成?
- (2) 将一个高级语言源程序转换成计算机能直接执行的机器代码通常需要哪几个步骤?
- (3) IA-32/x86-64 中的逻辑运算指令如何生成标志位信息? 移位指令可能会改变哪些标志位?
- (4) 执行条件跳转指令时所用到的标志信息从何而来? 请举例说明。
- (5) 无条件跳转指令和调用指令的相同点和不同点是什么?

3. 对于以下 AT&T 格式汇编指令, 根据操作数的长度确定对应指令助记符中的长度后缀, 并说明每个操作数的寻址方式。

- (1) `mov 8(%ebp, %ebx, 4), %ax`
- (2) `mov %al, 12(%ebp)`
- (3) `add(, %ebx, 4), %eax`
- (4) `or(%ebx), %dh`
- (5) `push %rcx`



```
(6) mov    $0xFFF0, %eax
(7) test   %rax, %rax
(8) lea     8(%ebx, %esi), %eax
```

4. 使用汇编器处理以下 IA-32/x86-64 中 AT&T 格式代码时都会产生错误，请说明每一行代码存在什么错误。

```
(1) movl    0xFF, (%eax)
(2) movb     %ax, 12(%ebp)
(3) addl     %ecx, $0xF0
(4) orw      $0xFFFF0, (%ebx)
(5) addb     $0xF8, (%dl)
(6) movl     %bx, %eax
(7) andl     %esi, %esx
(8) movq     8(%ebp, , 4), %rax
(9) leaq     20(%rdi, %rsi), %eax
```

5. 假设变量 x 和 ptr 的类型声明如下：

```
src_type x;
dst_type *ptr;
```

这里，src_type 和 dst_type 是用 typedef 声明的数据类型。有以下 C 语言赋值语句：

```
*ptr=(dst_type) x;
```

若 x 在寄存器 EAX 或 AX 或 AL 中，ptr 在寄存器 EDX 中，则对于表 3.11 中给出的 src_type 和 dst_type 类型组合，写出实现上述赋值语句的 IA-32 机器级代码，要求用 AT&T 格式汇编指令表示。

表 3.11 题 5 表

src_type	dst_type	机器级表示
char	int	
int	char	
int	unsigned	
short	int	
unsigned char	unsigned	
char	unsigned	
int	int	

6. 假设变量 x 和 y 分别存放在寄存器 RAX 和 RCX 中，给出以下各指令执行后寄存器 RDX 中的结果。

```
(1) leaq (%rax), %rdx
(2) leaq 4(%rax, %rcx), %rdx
(3) leaq (%rax, %rcx, 8), %rdx
(4) leaq 16(%rcx, %rax, 2), %rdx
(5) leaq (, %rax, 4), %rdx
(6) leaq (%rax, %rcx), %rdx
```

7. 假设在 IA-32 系统中以下地址以及寄存器中存放的机器数如表 3.12 所示。

不思考. 帮助理解

表 3.12 题 7 表

地址	机器数	寄存器	机器数
0x0804 9300	0xffff fff0	EAX	0x0804 9300
0x0804 9400	0x8000 0008	EBX	0x0000 0100
0x0804 9384	0x80f7 ff00	ECX	0x0000 0010
0x0804 9380	0x908f 12a8	EDX	0x0000 0080

分别说明执行以下指令后，哪些存储单元地址或寄存器中的内容会发生改变？改变后的内容是什么？

么？条件标志 OF、SF、ZF 和 CF 会发生什么改变？

- (1) `addl (%eax), %edx`
- (2) `subl (%eax, %ebx), %ecx`
- (3) `orw 4(%eax, %ecx, 8), %bx`
- (4) `testb $0x80, %dl`
- (5) `imull $32, (%eax, %edx), %ecx`
- (6) `mulw %bx`
- (7) `decw %cx`



8. 已知 IA-32 采用小端方式，根据给出的 IA-32 代码反汇编结果（部分信息用 x...x 表示）回答问题。

解. 3. 11

- (1) 已知 `je` 指令的操作码为 0111 0100，`je` 指令的跳转目标地址是什么？`call` 指令中的跳转目标地址 0x80483b1 是如何反汇编出来的？

```
804838c: 74 08                je      xxxxxxxx
804838e: e8 1e 00 00 00      call   0x80483b1<test>
```

- (2) 已知 `jb` 指令的操作码为 0111 0010，`jb` 指令的跳转目标地址是什么？`movl` 指令中的目的地址如何反汇编出来的？

```
8048390: 72 f6                jb      xxxxxxxx
8048392: c6 05 00 a8 04 08 01 movl    $0x1, 0x804a800
8048399: 00 00 00
```

- (3) 已知 `jle` 指令的操作码为 0111 1110，`jle` 和 `mov` 指令的地址各是什么？

```
xxxxxxx: 7e 16                jle     0x80492e0
xxxxxxx: 89 d0                mov     %edx, %eax
```

- (4) 已知 `jmp` 指令的跳转目标地址采用相对寻址方式。`jmp` 指令操作码为 1110 1001，其跳转目标地址是什么？

```
8048296: e9 00 ff ff ff jmp     xxxxxxxx
804829b: 29 c2                sub     %eax, %edx
```

9. 对于以下 x86-64 中的 AT&T 格式汇编指令，根据操作数的长度确定对应指令助记符中的长度后并说明每个操作数的寻址方式。

- (1) `mov 8(%rbp, %rbx, 4), %ax`
- (2) `mov %al, 12(%rbp)`
- (3) `add (%rbp, %rsi, 4), %r9w`
- (4) `or (%rbx), %dil`
- (5) `sub %bpl, 8(%rsp, %rdi, 4)`
- (6) `mov $0xFFFF0, %eax`
- (7) `test %r8d, %r8d`
- (8) `lea 8(%rbx, %rsi), %rax`
- (9) `xor $0xF0, %rax`



10. 假设 x86-64 系统某程序中变量 x 和 ptr 的类型声明如下：

```
src_type x;  
dst_type *ptr;
```

这里，src_type 和 dst_type 是用 typedef 声明的数据类型。有以下几个 C 语言赋值语句：

```
*ptr=(dst_type) x;
```

若 x 存储在寄存器 RAX 或 EAX 或 AX 或 AL 中，ptr 存储在寄存器 RDX 中，则对于表 3.13 给出的 src_type 和 dst_type 的类型组合，写出实现上述赋值语句对应的汇编指令。

表 3.13 题 10 表

src_type	dst_type	汇编指令（AT&T 格式）
char	long	
int	char	
int	unsigned long	
short	int	
unsigned char	unsigned	
char	unsigned long	
unsigned long	int	

习题

1. 给出以下概念的解释说明。

过程调用	调用约定	非静态局部变量	现场信息
栈 (stack)	ABI 规范	叶子过程	当前栈帧
调用者保存寄存器	被调用者保存寄存器	帧指针寄存器	按值传递参数
按地址传递参数	嵌套调用	递归调用	栈溢出
缓冲区溢出	缓冲区溢出攻击	栈随机化	金丝雀值

2. 简单回答下列问题。

- (1) 按值传递参数和按地址传递参数两种方式有哪些不同点?
- (2) 为什么在递归深度较深时递归调用的时间开销和空间开销都会较大?
- (3) 对于 auto 型变量, 编译器如何分配其空间? 通常被分配在什么存储区? 对于 auto 型变量地址进行大小比较, 是否有意义? 为什么?
- (4) 为什么数据在存储器中最好按对齐方式存放?
- (5) 有哪几种防止缓冲区溢出攻击的基本方法?

3. 假设某个 C 语言函数 func() 的原型声明如下:

```
void func(int *xptr, int *yptr, int *zptr);
```

函数 func() 的过程体对应的机器级代码用 AT&T 汇编形式表示如下:

```
1 movl 8(%ebp), %eax
2 movl 12(%ebp), %ebx
3 movl 16(%ebp), %ecx
4 movl (%ebx), %edx
5 movl (%ecx), %esi
6 movl (%eax), %edi
7 movl %edi, (%ebx)
8 movl %edx, (%ecx)
9 movl %esi, (%eax)
```

回答下列问题或完成下列任务。

- (1) 上述机器级代码是在 IA-32 系统还是 x86-64 系统中生成的? 为什么?
- (2) 在过程体开始时, 三个入口参数对应实参所存放的存储单元地址是什么? (提示: 当前栈帧底部由帧指针寄存器 EBP 指示。)
- (3) 根据上述机器级代码写出函数 func() 的 C 语言代码。

4. 假设函数 operate() 的部分 C 语言代码如下:

```
1 int operate(int x, int y, int z, int k) {
2     int v = _____ ;
3     return v;
4 }
```

以下 IA-32 汇编代码用来实现第 2 行语句的功能:

```
1 movl 12(%ebp), %ecx
2 sall $8, %ecx
```



```

3 movl 8(%ebp), %eax
4 movl 20(%ebp), %edx
5 imull %edx, %eax
6 movl 16(%ebp), %edx
7 andl $65520, %edx
8 addl %ecx, %edx
9 subl %edx, %eax

```

不考。考根据汇编补充C

回答下列问题或完成下列任务。

- (1) 写出每条汇编指令的注释，并填写 operate() 函数中缺失的部分。
 - (2) 给出函数 operate() 对应的 x86-64 汇编代码，并和 IA-32 汇编代码进行性能比较。
5. 假设函数 product() 的 C 语言代码如下，其中 num_type 是用 typedef 声明的数据类型。

```

1 void product(num_type *d, unsigned x, num_type y) {
2     *d = x*y;
3 }

```

函数 product() 的过程体对应的 IA-32 汇编代码如下：

```

1 movl 12(%ebp), %eax
2 movl 20(%ebp), %ecx
3 imull %eax, %ecx
4 mull 16(%ebp)
5 leal (%ecx, %edx), %edx
6 movl 8(%ebp), %ecx
7 movl %eax, (%ecx)
8 movl %edx, 4(%ecx)

```

给出上述每条汇编指令的注释，并说明 num_type 是什么类型。

6. 已知函数 comp() 的 C 语言代码及其过程体对应的汇编代码如图 4.28 所示。回答下列问题或完成下列任务。

- (1) 图中给出的是 IA-32 还是 x86-64 对应的汇编代码？为什么？
- (2) 给出每条汇编指令的注释，并说明为什么 C 语言代码中只有一个 if 语句而汇编代码中有两条跳转指令。

<pre> 1 void comp(char x, int *p) { 2 3 if (p && x<0) 4 *p += x; 5 } </pre>	<pre> 1 testq %rsi, %rsi 2 je .L1 3 testb \$0x80, %dil 4 jns .L1 5 movsbl %dil, %edi 6 addl %edi, (%rsi) 7 .L1: </pre>
--	--

图 4.28 题 6 图

7. 已知函数 func() 的 C 语言代码框架及其过程体对应的 IA-32 汇编代码如图 4.29 所示，根据对应的汇编代码填写 C 语言代码中缺失的表达式。
8. 已知函数 do_loop() 的 C 语言代码如下：

```

1 short do_loop(short x, short y, short k) {
2     do {

```



```
3      x*=(y&k) ;
4      k--;
5  } while ((k>0) && (y>k));
6  return x;
7 }
```

```
1 int func(int x, int y) {
2
3     int z = _____;
4     if ( _____ ) {
5         if ( _____ )
6             z = _____ ;
7         else
8             z = _____ ;
9     } else if ( _____ )
10         z = _____ ;
11     return z;
12 }
```

```
1  movl    8(%ebp), %eax
2  movl    12(%ebp), %edx
3  cmpl    $-100, %eax
4  jg      .L1
5  cmpl    %eax, %edx
6  jle     .L2
7  addl    %edx, %eax
8  jmp     .L3
9  .L2:
10 subl    %edx, %eax
11 jmp     .L3
12 .L1:
13 cmpl    $16, %eax
14 jl      .L4
15 andl    %edx, %eax
16 jmp     .L3
17 .L4:
18 imull    %edx, %eax
19 .L3:
```

图 4.29 题 7 图

函数 do_loop() 的过程体对应的 IA-32 汇编代码如下：

```
1  movw    8(%ebp), %bx
2  movw    12(%ebp), %si
3  movw    16(%ebp), %cx
4  .L1:
5  movw    %si, %dx
6  movw    %dx, %ax
7  sarw    $15, %dx
8  idiv    %cx
9  imulw    %dx, %bx
10 decw    %cx
11 testw    %cx, %cx
12 jle     .L2
13 cmpw    %cx, %si
14 jg      .L1
15 .L2:
16 movswl    %bx, %eax
```

回答下列问题或完成下列任务。

- (1) 给每条汇编指令添加注释，并说明每条指令执行后，目的寄存器中存放的是什么内容？
- (2) 上述函数过程体中用到了哪些被调用者保存寄存器和哪些调用者保存寄存器？在该函数过程体前面的准备阶段，哪些寄存器必须保存到栈中？
- (3) 为什么第 7 行中的 DX 寄存器需要算术右移 15 位？
- (4) 给出 do_loop() 函数对应的 x86-64 汇编代码。

9. 已知函数 f1() 的 C 语言代码框架及对应的 x86-64 汇编代码如图 4.30 所示，根据汇编代码填写 C 语言代码中缺失部分，并说明函数 f1() 的功能。

```
1 int f1(unsigned x) {
2
3     int y = 0 ;
4     while (_____) {
5         _____ ;
6     }
7     return _____ ;
8 }
```

```
1  movl    $0, %eax
2  testl   %edi, %edi
4  je      .L1
5  .L2:
6  xorl    %edi, %eax
7  shrl    $1, %edi
8  jne     .L2
9  .L1:
10 andl    $1, %eax
```

图 4.30 题 9 图

10. 已知函数 sw() 的 C 语言代码框架如下：

```
int sw(int x) {
    int v=0;
    switch (x) {
        /* switch 语句中的处理部分省略 */
    }
    return v;
}
```

函数 sw() 过程体中开始部分的 IA-32 汇编代码以及跳转表如图 4.31 所示。

```
1  movl    8(%ebp), %eax
2  addl    $3, %eax
3  cmpl    $7, %eax
4  ja      .L7
5  jmp     *.L8( , %eax, 4)
6  .L7:
7  ...
8  ...
```

```
1  .L8:
2      .long    .L7
3      .long    .L2
4      .long    .L2
5      .long    .L3
6      .long    .L4
7      .long    .L5
8      .long    .L7
9      .long    .L6
```

图 4.31 题 10 图

- 回答下列问题。
- (1) 函数 sw() 中的 switch 语句处理部分标号的取值情况如何？
 - (2) 标号的取值在什么情况下执行 default 分支？哪些标号的取值会执行同一个 case 分支？

11. 已知 C 语言函数 test() 的入口参数有 a、b、c 和 p，过程体代码如下：

```
*p = a;
return b*c;
```

函数 test() 过程体对应的 IA-32 汇编代码如下：

```
1  movl    20(%ebp), %edx
2  movsbw  8(%ebp), %ax
3  movw    %ax, (%edx)
4  movzwl  12(%ebp), %eax
5  movzwl  16(%ebp), %ecx
```

```
6  mull    %ecx
```

完成下列任务。

- (1) 写出函数 test() 的原型，以给出返回参数的类型以及入口参数 a、b、c 和 p 的类型和顺序。
- (2) 写出对应的 x86-64 汇编代码。

12. 已知函数 funct() 的 C 语言代码如下：

```
1  #include <stdio.h>
2  int funct(void) {
3      int x, y;
4      scanf("%d %d", &x, &y);
5      return x-y;
6  }
```

函数 funct() 对应的 IA-32 汇编代码如下：

```
1  funct:
2      pushl    %ebp
3      movl     %esp, %ebp
4      subl     $40, %esp
5      leal     -8(%ebp), %eax
6      movl     %eax, 8(%esp)
7      leal     -4(%ebp), %eax
8      movl     %eax, 4(%esp)
9      movl     $.LC0, (%esp)      # 将指向字符串 "%d %d" 的指针入栈
10     call     scanf              # 假定 scanf 执行后 x=15, y=20
11     movl     -4(%ebp), %eax
12     subl     -8(%ebp), %eax
13     leave
14     ret
```

假设函数 funct() 开始执行时，R[esp]=0xbc00 0020，R[ebp]=0xbc00 0030，指向字符串 “%d %d” 的指针为 0x804 c000。回答下列问题或完成下列任务。

- (1) 执行第 3、10 和 13 行的指令后，寄存器 EBP 中的内容分别是什么？
- (2) 执行第 3、10 和 13 行的指令后，寄存器 ESP 中的内容分别是什么？
- (3) 局部变量 x 和 y 所在存储单元的地址分别是什么？
- (4) 画出执行第 10 行指令后 funct 的栈帧，给出栈帧中的内容及其地址。

13. 已知递归函数 refunc() 的 C 语言代码框架如下：

```
1  int refunc(unsigned x) {
2      if ( _____ )
3          return _____ ;
4      unsigned nx = _____ ;
5      int rv = refunc(nx) ;
6      return _____ ;
7  }
```

上述递归函数过程体对应的 IA-32 汇编代码如下：

```
1  movl     8(%ebp), %ebx
2  movl     $0, %eax
```



```
3    testl    %ebx, %ebx
4    je       .L2
5    movl     %ebx, %eax
6    shrl     $1, %eax
7    movl     %eax, (%esp)
8    call     refunc
9    movl     %ebx, %edx
10   andl     $1, %edx
11   leal     (%edx, %eax), %eax
12 .L2:
```

根据对应的汇编代码填写 C 语言代码中缺失的部分，并说明函数的功能。

14. 针对 IA-32 和 x86-64 两种系统，填写表 4.4，说明每个数组元素的大小、整个数组的大小以及第*i*个元素的地址。

表 4.4 题 14 表

数组	元素大小 /B	数组大小 /B	起始地址	元素 <i>i</i> 的地址
int A[10]			&A[0]	
long B[100]			&B[0]	
short*C[5]			&C[0]	
short**D[6]			&D[0]	
long double E[10]			&E[0]	
long double *F[10]			&F[0]	

15. 假设在 x86-64 系统中，short 型数组 S 的首地址 AS 和数组下标（索引）变量 *i* 分别存放在寄存器 RDX 和 RCX 中，表 4.5 给出的表达式的结果存放在 RAX 或 AX 中，仿照例子填写表 4.5，说明表达式的类型、值和相应的汇编代码。

表 4.5 题 15 表

表达式	类型	值	汇编代码
S			
S+i-3			
S[i]	short	M[AS+2*i]	movw (%rdx, %rcx, 2), %ax
&S[10]			
&S[i+2]	short *	AS+2*i+4	leaq 4(%rdx, %rcx, 2), %rax
&S[i]-S			
S[4*i+4]			
*(S+i-2)			

16. 假设函数 sumij() 的 C 语言代码如下，其中，M 和 N 是用 #define 声明的常数。

```
1  int a[M][N], b[N][M];
2
3  int sumij(int i, int j) {
4      return a[i][j] + b[j][i];
5  }
```

已知函数 sumij() 的 IA-32 过程体对应的汇编代码如下：

```

1  movl    8(%ebp), %ecx
2  movl    12(%ebp), %edx
3  leal    (, %ecx, 8), %eax
4  subl    %ecx, %eax
5  addl    %edx, %eax
6  leal    (%edx, %edx, 4), %edx
7  addl    %ecx, %edx
8  movl    a(, %eax, 4), %eax      # a 表示数组 a 的首地址
9  addl    b(, %edx, 4), %eax      # b 表示数组 b 的首地址

```

根据上述汇编代码，确定 M 和 N 的值。

17. 假设函数 `st_ele()` 的 C 语言代码如下，其中，L、M 和 N 是用 `#define` 声明的常数。

```

1  int a[L][M][N];
2
3  int st_ele(int i, int j, int k, int *dst) {
4      *dst = a[i][j][k];
5      return sizeof(a);
6  }

```

已知函数 `st_ele()` 的过程体对应的 IA-32 汇编代码如下：

```

1  movl    8(%ebp), %ecx
2  movl    12(%ebp), %edx
3  leal    (%edx,%edx, 8), %edx  8j
4  movl    %ecx, %eax
5  sall    $6, %eax  i * 2^6 -> 64i
6  subl    %ecx, %eax  63i
7  addl    %eax, %edx  8j + 63i
8  addl    16(%ebp), %edx  8j + 63i + k
9  movl    a(, %edx, 4), %eax      # a 表示数组 a 的首地址
10 movl    20(%ebp), %edx
11 movl    %eax, (%edx)
12 movl    $4536, %eax
    sizeof(a)

```

$a + 4 * (63i + 8j + k)$
 $a[4 * \%edx] \rightarrow \%eax$ 即 $a[i][j][k]$

根据上述汇编代码确定 L、M 和 N 的值，并写出函数 `st_ele()` 对应的 x86-64 汇编代码。

18. 假设函数 `trans_matrix()` 的 C 语言代码如下，其中，M 是用 `#define` 声明的常数。

```

1  void trans_matrix(int a[M][M]) {
2      int i, j, t;
3      for (i = 0; i < M; i++)
4          for (j = 0; j < M; j++) {
5              t = a[i][j];
6              a[i][j] = a[j][i];
7              a[j][i] = t;
8          }
9  }

```

已知采用优化编译（选项 `-O2`）后函数 `trans_matrix()` 的内循环对应的 IA-32 汇编代码如下：

```

1  .L2:
2  movl    (%ebx), %eax
3  movl    (%esi, %ecx, 4), %edx

```



```
4    movl    %eax, (%esi, %ecx, 4)
5    addl    $1, %ecx
6    movl    %edx, (%ebx)
7    addl    $76, %ebx
8    cmpl    %edi, %ecx
9    jl      .L2
```

根据上述汇编代码，回答下列问题或完成下列任务。

- (1) M 的值是多少？常数 M 和变量 j 分别存放在哪个寄存器中？
- (2) 写出上述优化汇编代码对应的函数 trans_matrix() 的 C 代码。

19. 假设结构体类型 node 的定义、函数 np_init() 部分 C 语言代码及对应的 IA-32 部分汇编代码如图 4.32 所示。

```
struct node {
    int *p;
    struct {
        int x;
        int y;
    } s;
    struct node *next;
};
```

```
void np_init(struct node *np)
{
    np->s.x = _____ ;
    np->p = _____ ;
    np->next= _____ ;
}
```

```
movl 8(%ebp), %eax
movl 8(%eax), %edx
movl %edx, 4(%eax)
leal 4(%eax), %edx
movl %edx, (%eax)
movl %eax, 12(%eax)
```

图 4.32 题 19 图

回答下列问题或完成下列任务。

- (1) 结构体 node 所需存储空间为多少字节？成员 p、s.x、s.y 和 next 的偏移地址分别为多少？
- (2) 根据汇编代码填写 np_init() 中缺失的表达式。
- (3) 写出图 4.32 中 IA-32 汇编代码对应的 x86-64 汇编代码。

20. 假设联合体类型 utype 的定义如下：

```
typedef union {
    struct {
        int    x;
        short y;
        short z;
    } s1;
    struct {
        short a[2];
        int    b;
        char   *p;
    } s2;
} utype;
```

若存在具有如下形式的一组函数：

```
void getvalue(utype *uptr, TYPE *dst) {
    *dst = EXPR;
}
```

该组函数用于计算不同表达式 EXPR 的值，返回值的数据类型根据表达式的类型确定。假设函数 getvalue() 的入口参数 uptr 和 dst 分别被装入寄存器 EAX 和 EDX 中，仿照例子填写表 4.6，说明

在不同表达式下的 TYPE 类型以及表达式对应的 IA-32 汇编指令序列（要求尽量只使用 EAX 和 EDX，不够用时再使用 ECX）。

表 4.6 题 20 表

表达式 EXPR	TYPE 类型	汇编指令序列
uptr->s1.x	int	movl (%eax), %eax movl %eax, (%edx)
uptr->s1.y		
&uptr->s1.z		
uptr->s2.a		
uptr->s2.a[uptr->s2.b]		
*uptr->s2.p		

21. 分别给出在 IA-32+Linux、x86-64+Linux 平台下，下列各个结构体类型中每个成员的偏移量、结构体总大小以及结构体起始位置的对齐要求。

- (1) struct S1 {short s; char c; int i; char d;;}
- (2) struct S2 {int i; short s; char c; char d;;}
- (3) struct S3 {char c; short s; int i; char d;;}
- (4) struct S4 {short s[3]; char c; ;}
- (5) struct S5 {char c[3]; short *s; int i; char d; double e;;}
- (6) struct S6 {struct S1 c[3]; struct S2 *s; char d;;}

22. 以下是结构体 test 的声明：

```
struct {
    char      c;
    double    d;
    int       i;
    short     s;
    char      *p;
    long      l;
    long long g;
    void      *v;
} test;
```

假设在 Windows 平台上编译，则这个结构体中每个成员的偏移量是多少？结构体总大小为多少字节？如何调整成员的先后顺序使结构体所占空间最小？

23. 图 4.33 给出了函数 getline() 存在漏洞和问题的 C 语言代码实现，右边是其对应的 IA-32 反汇编部分结果。

```
char *getline() {
    char buf[8];
    char *result;
    gets(buf);
    result=malloc(strlen
        (buf));
    strcpy(result, buf);
    return result;
}
```

```
1 0804840c <getline>:
2 804840c: 55                push    %ebp
3 804840d: 89 e5            mov     %esp, %ebp
4 804840f: 83 ec 28        sub     $0x28, %esp
5 8048412: 89 5d f4        mov     %ebx, -0xc(%ebp)
6 8048415: 89 75 f8        mov     %esi, -0x8(%ebp)
7 8048418: 89 7d fc        mov     %edi, -0x4(%ebp)
8 804841b: 8d 75 ec        lea     -0x14(%ebp), %esi
9 804841e: 89 34 24        mov     %esi, (%esp)
10 8048421: e8 a3 ff ff ff  call    80483c9 <gets>
```

图 4.33 题 23 图